

dkr_Portfolio: Design and Implementation of a Personal Portfolio Web Application

Dheeraj Kumar
IIT KHARAGPUR
dheeraj6122002@gmail.com

2026-09-06

Correspondence: dheeraj6122002@gmail.com · Affiliation: IIT KHARAGPUR · ORCID: <https://orcid.org/0009-0007-3742-8341>

Abstract

We present dkr_Portfolio, an open-source personal portfolio system that replaces manual mark-down authoring with an edge-mediated, AI-assisted publishing pipeline. The site is a statically-exported Next.js application deployed on GitHub Pages; content is created by pasting free-text Raw Notes into an Admin Portal modal, which drives a two-phase interview with a self-hosted large language model (LLM) accessed through a Cloudflare Worker and a Cloudflare Tunnel. When notes lack architectural detail, metrics, or STAR structure, the model returns targeted clarifying questions; otherwise it emits a validated Portfolio Entry with strict YAML frontmatter and a Situation/Task/Action/Result body. The worker commits the entry to the repository via the GitHub REST API, after which GitHub Actions validates, builds, and publishes the site. The design combines static-site discipline, edge computing, human-in-the-loop machine learning, and continuous software engineering in a fully self-hostable package released under AGPL-3.0. We describe the architecture, the content contract, the edge protocol, and the operational trade-offs, grounding each choice in the relevant literature.

1 Introduction

Personal portfolios rot when publishing costs exceed the motivation to publish. The conventional loop – write markdown, satisfy a frontmatter schema, commit, watch CI – is simple for engineers but still manual enough that project notes accumulate as unstructured text and never become entries. dkr_Portfolio inverts the burden: the author pastes Raw Notes and answers only the questions an LLM judge deems necessary, and the system handles structuring, validation, commit, build, and deploy.

The system as deployed at commit 4fa1ba4 consists of three cooperating layers. First, a statically-exported Next.js 15.5 site (output: export, basePath: /dkr_Portfolio, React 19, Tailwind 3.4) renders `_posts/*.md` as cards on a dark-glassmorphism grid, live at https://dheerajkumar1a1a.github.io/dkr_Portfolio/. Second, a Cloudflare Worker (backend-worker/src/index.js, ~430 lines) authenticates the author, calls a self-hosted Ollama model (glm-4.7-flash:latest, with an OpenRouter free-router fallback), enforces a strict JSON contract, and commits base64-encoded

markdown through the GitHub contents API. Third, a GitHub Actions workflow validates frontmatter, builds the export, and publishes to Pages in about 55 seconds.

Our contributions are practical rather than algorithmic: (i) a compositional portfolio-publishing architecture with cleanly separated site, edge, model, and CI concerns; (ii) a two-phase interview protocol that asks for missing detail only when needed yet always offers a skip-and-publish escape hatch; (iii) a strict-yet-lenient content contract – strict in CI, lenient at render – so one malformed file can neither blank the homepage nor silently ship; and (iv) a fully documented, zero-license-cost deployment that fits comfortably inside free tiers at portfolio scale.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the system architecture. Section 4 details the content model and pipeline methodology. Section 5 discusses implementation considerations. Section 6 discusses limitations and future work. Section 7 concludes.

2 Related Work

2.1 Web architecture and static sites

The modern Web succeeded because its software architecture matched Internet scale: scalable component interactions, general interfaces, independent deployability, and intermediaries for latency, security, and encapsulation (Fielding and Taylor 2002). Fielding and Taylor’s Representational State Transfer (REST) style both described the deployed Web and guided the redesign of HTTP and URIs. `dkr_Portfolio` follows this lineage directly: the site is a static export with no origin server, the edge worker is a stateless intermediary, and all mutations flow through the RESTful GitHub contents API (GET for sha, PUT for create/update). Modularity follows the classical criterion that decomposition should hide design decisions behind narrow interfaces (Parnas 1972): site rendering, edge orchestration, model inference, and delivery each change independently.

Literate authoring has an older pedigree. Knuth argued that programs should be works of literature addressed to humans first and machines second (Knuth 1984). Portfolio Entries apply the same inversion to project memory: the durable artifact is a human-readable markdown narrative with machine-checkable frontmatter, not a database row. The STAR body (Situation/Task/Action/Result) is the portfolio analogue of Knuth’s interwoven prose and code.

2.2 Edge computing

Shipping inference-adjacent orchestration to the edge trades a small, bursty compute footprint for global reach and zero server maintenance. Shi et al. surveyed the vision and challenges of edge computing – latency reduction, bandwidth savings, and the need to manage constrained, widely distributed execution (Shi et al. 2016). Our worker (12.08 KiB, 3.57 KiB gzipped per wrangler `deploy --dry-run`) fits this envelope exactly: CORS handling, password comparison, tunnel-URL normalization, prompt assembly, a 45-second abort-bounded model call, fence-stripping, JSON validation, frontmatter normalization, and a two-step GitHub commit. It holds no session state; idempotency comes from server-generated filenames (`YYYY-MM-DD-portfolio-update-<6hex>.md`) rather than client-supplied paths, which simultaneously defeats path traversal even if the model hallucinates a path.

2.3 Continuous delivery and GitHub as infrastructure

Continuous software engineering treats business, development, and operations as one cadence from planning to deployment (Fitzgerald and Stol 2017). The repository implements the canonical loop at portfolio scale: `checkout@v4`, `setup-node@v4` (Node 22), `npm ci`, `validate:posts`, `next build`, `upload-pages-artifact@v3` to `deploy-pages@v4`. Validation runs before the build so a malformed entry fails fast (as observed in the project’s own history when a bad date failed the workflow and a follow-up fix restored it). Kalliamvakou et al. cautioned that GitHub repositories are not a neutral mirror of practice – most projects are personal, inactive, or non-software, and activity concentrates in a minority (Kalliamvakou et al. 2014). `dkr_Portfolio` embraces that finding: it is unapologetically a personal, low-traffic system whose automation (server-side filenames, sha-aware PUT, `branch: main` commit messages of the form `feat(blog): add ... via edge worker`) is tuned for a few publishes per week rather than organizational throughput.

Process-wise the project follows an iterative, risk-driven spiral rather than a single waterfall pass (Boehm 1988): the `n8n` webhook predecessor was replaced by the Worker (ADR-0001), the Ollama integration moved from an OpenAI-compatible `/v1` shim to the native `/api/chat` endpoint (ADR-0002), and the content contract was hardened from loose MDX to strict `_posts/*.md` with CI lint (ADR-0003), each cycle retiring one operational risk.

2.4 Human-in-the-loop synthesis with language models

Interactive machine learning succeeds only when the system accounts for the user as tightly as for the loss function: interactivity couples system and user, and systems that ignore the human side fail in predictable ways (Amershi et al. 2014). The Admin Portal is designed around that coupling. Phase 1 collects Raw Notes plus an admin password and an optional per-request Ollama URL; Phase 2 presents 3–5 clarifying questions only when the model judges the notes insufficient, and always offers Skip Questions & Deploy Anyway, which appends a skip directive forcing `needs_more_info=false` and generates a best-effort entry with N/A placeholders. Empty answers are normalized to "N/A" server-side, and double-submit is guarded both by a disabled button and an early return.

The synthesis step relies on two widely replicated LLM behaviors. First, prompting a model to reason explicitly before answering elicits substantially better multi-step behavior than answer-only prompts (Wei et al. 2022); our system prompt front-loads the STAR and frontmatter specification so the model’s JSON decision (`needs_more_info`, `questions`, `final_markdown`) is already a reasoned completeness judgment. Second, models of code and structured text generalize from few examples without gradient updates (Madaan et al. 2022), which is why a single system prompt plus the author’s notes suffice – no fine-tuning is involved. Instruction tuning on curated feedback is what makes such zero-shot obedience reliable in open-weight models (Li et al. 2023); we consume that capability rather than reproduce it, pinning `glm-4.7-flash:latest` on the demo host with `llama3.1:8b` and `qwen2.5:14b` as documented fallbacks.

3 System Architecture

The pipeline has four principals – browser modal, edge worker, model endpoint, and GitHub – arranged so that secrets never reach the client and the client never chooses a file path.

```

[ AdminPortal.tsx Ctrl+Shift+A ] -- POST {password, rawNotes, clarifications?,
skipQuestions?, ollamaUrl?} --> [ Edge Worker portfolio-pipeline-
worker.prtf.workers.dev ]
|
|
|   | <-- {status:"needs_answers",questions} / {status:"success",filename} /
{status:"error",message} -----|
|
|
|   |                                     (- POST /api/chat
format:json think:false, 45s timeout -)-----+
|   |                                     v
|   |                                     [ Local Ollama glm-4.7-flash:latest
via Cloudflare Tunnel ]
|   |                                     ^ returns JSON
{needs_more_info, questions, final_markdown}
|
|
|   +------(PUT /repos/.../contents/_posts/*.md
base64, branch:main, sha?)-----> [ GitHub REST API ]
|
v
[ GitHub Actions deploy.yml ]
|
v
[ GitHub Pages https://dheerajkumarlala.github.io/dkr_Portfolio/ ]

```

Figure 1. Request flow through the portfolio publishing pipeline.

The modules are:

- **src/components/AdminPortal.tsx** – Ctrl+Shift+A modal (Esc closes). Captures rawNotes, admin password, optional per-request ollamaUrl matching `https://*/v1`, and Phase-2 clarifications: {question, answer}[] .Two Phase-2 actions: Submit Answers & Deploy and amber Skip Questions & Deploy Anyway (sends N/A clarifications plus skipQuestions:true).
- **backend-worker/src/index.js** – edge handler (~430 lines): CORS allowlist (`https://dheerajkumarlala.github.io`, `http://localhost:3000`, Vary: Origin), method gate (POST only, OPTIONS preflight), provider resolution (client Ollama URL takes precedence, then client OpenRouter key, then server OLLAMA_TUNNEL_URL, then server OPENROUTER_API_KEY), password check against ADMIN_PASSWORD, prompt assembly, model call, stripFences + JSON.parse + shape validation, normalizeFrontmatter, server-side filename generation, GET-for-sha (cache:no-store) then PUT with TextEncoder-to-base64 content.
- **src/lib/api.ts** – build-time compiler: reads `_posts/*.md` (files ending .md only), requires opening ---, parses with gray-matter, enforces non-empty title/summary/non-empty techStack, slices ISO datetimes to YYYY-MM-DD via parseFrontmatterDate (falling back to file

birthtime), renders body with remark + remark-html, sorts descending by date. Malformed files log a warning and are skipped, so the homepage shows “Awaiting Data” only when zero entries are valid.

- **scripts/validate-posts.mjs** – CI gate (Node + gray-matter): fails the workflow on missing delimiters, missing fields, or bad dates, accepting both quoted strings and js-yaml-parsed Date objects.
- **.github/workflows/deploy.yml** – checkout, Node 22, npm ci, validate, next build (output: export), artifact upload and Pages deploy.
- **docs/adr/** – three decision records: 0001 (Worker replaces the legacy n8n workflow JSON kept for reference), 0002 (Ollama via tunnel native chat), 0003 (canonical _posts/*.md frontmatter contract; legacy content/projects/*.mdx deleted as dead code).
- **_posts/** – eight validated entries at the time of writing (six dated 2026-08-24 through 2026-09-01 from the worker rollout plus two 2026-09-02 edge-worker commits), each a STAR narrative under a quoted frontmatter header.

4 Methodology

4.1 Content model

A Portfolio Entry is a single _posts/*.md file with mandatory --- delimiters:

```
---
title: "Automated Kaggle Notebook Documentation Pipeline"
date: "2024-06-23"
techStack: ["Python", "Hugo", "Docker"]
summary: "Built a Python pipeline to automate documentation..."
---

## Situation
...

## Task
...

## Action
1. ...

## Result
...
```

Field	Type	Notes
title	quoted string	Human title, never kebab-case
date	quoted "YYYY-MM-DD"	Worker regex-normalizes 2024-06-23T00:00:00.000Z and unquoted forms to the quoted date
techStack	string[]	1–8 items
summary	quoted string	One sentence, under 200 characters

The contract is deliberately redundant: the model is instructed to emit quoted dates, the worker rewrites them with `normalizeFrontmatter`, the validator accepts strings, ISO prefixes, and `Date` objects, and the renderer slices defensively. Agreement across four layers is what makes AI-generated YAML safe to commit blind.

4.2 Two-phase interview protocol

Phase 1 sends `{password, rawNotes, ollamaUrl?}`. The worker builds a system prompt demanding exactly `{needs_more_info: boolean, questions: string[] | null, final_markdown: string | null}` with no fences and no prose, plus a user message of Raw Notes (and Clarifications on the second pass). If the model judges the notes insufficient, the worker returns `{status:"needs_answers", questions}` with 3–5 questions (e.g., which versions, what metrics, what was the author’s specific contribution). Phase 2 returns `{password, rawNotes, clarifications, skipQuestions?}`. The skip path appends a `CRITICAL: SKIP` directive that forces markdown generation from available information. Either way the terminal states are `{status:"success", filename}` or `{status:"error", message}` from a closed taxonomy: 401 unauthorized, 400 missing fields or bad `ollamaUrl`, 502 tunnel offline / malformed model output / GitHub commit failure.

4.3 Edge worker internals

Provider resolution is client-first: an explicit per-request Ollama URL (useful for pointing at a colleague’s tunnel or a new model without redeploy) wins over the server `OLLAMA_TUNNEL_URL` secret, and an explicit OpenRouter key is honored even when an Ollama tunnel is configured, fixing the failure mode where a stale tunnel masked a valid fallback. The Ollama call strips any `/v1` or `/v1/chat/completions` suffix, appends `/api/chat`, and posts `{model, format:"json", stream:false, think:false, messages:[system, user]}` with `AbortSignal.timeout(45000)`. The OpenRouter fallback posts to `https://openrouter.ai/api/v1/chat/completions` with `model: OPENROUTER_MODEL || "openrouter/free"`, `response_format:{type:"json_object"}`, and referrer headers, mapping 401/402/429/404 to actionable hints. Model output is fence-stripped, parsed, and shape-checked (`needs_more_info` must be boolean; `questions` must be non-empty when requested; `final_markdown` must contain `---`); any deviation yields AI returned malformed structured output without leaking tunnel URLs or tokens.

The GitHub commit is a read-then-write: `GET .../contents/_posts/{filename}` with `cache:no-store` distinguishes create (404) from update (200 with sha); the PUT carries content as UTF-8-safe base64 (TextEncoder to btoa, so emojis survive), `branch: main`, and `message: feat(blog): add {filename} via edge worker`. GitHub response bodies are never forwarded to the client.

4.4 Validation: strict CI, lenient runtime

`validate-posts.mjs` checks delimiters, required fields, string types, date shape, and non-empty `techStack`, printing one OK line per file. `api.ts` enforces the same required fields but degrades gracefully per file. The combination means a bad commit fails loudly in Actions (where it can be fixed and pushed) while a bad file that somehow reaches production can never blank the site. Both layers handle the `js-yaml` quirk where an unquoted date parses as a `Date`.

4.5 Deployment and cost envelope

The worker deploys with `wrangler deploy` (free tier: 100k requests/day, 10 ms CPU, `workers.dev` subdomain; measured upload 12.08 KiB). Secrets (`ADMIN_PASSWORD`, `GITHUB_PAT` with `Contents: Read & Write`, `OLLAMA_TUNNEL_URL`, `OLLAMA_MODEL`) are set via `wrangler secret put`, each triggering a redeploy. The site deploys via Pages with Source: GitHub Actions; `next.config.mjs` sets `output: export` and `basePath: /dkr_Portfolio`, which does not affect the absolute `WORKER_URL`. Observed performance on the demo host is ~1.8 s for vague notes (`fast needs_answers`) and ~14.7 s for detailed notes through `glm-4.7-flash`, a ~55 s Actions build, Pages live in 2–3 minutes, and a ~100 kB first-load out/ bundle. The source is AGPL-3.0: free to self-host and modify, with network copyleft on the Corresponding Source.

5 Implementation Considerations

Three decisions carry most of the weight. First, filenames are server-controlled randomness, not model output: this single choice eliminates path traversal, guarantees uniqueness without a round-trip, and keeps the PUT idempotent. Second, authentication is a shared secret compared with a timing-safe-enough strict equality on the edge, never bundled to the client except as user input; the PAT lives only in worker secrets and the Actions ephemeral token handles deploys. Third, the model is treated as an unreliable narrator inside a reliable harness: every byte it returns passes through fence-stripping, JSON parsing, shape checks, frontmatter normalization, and finally the CI validator before it can reach readers. The harness, not the prompt, is the correctness boundary.

Self-hosting the model via a Cloudflare Tunnel (quick `trycloudflare.com` URLs for development, named tunnels for production) removes per-token cost and keeps Raw Notes off third-party APIs, at the price of tunnel ephemerality and VRAM floors (roughly 8 GB for an 8B Q4 model, 16 GB for 14B, ~20 GB for the 29.9B `glm-4.7-flash` on the demo host). The OpenRouter fallback exists precisely for the window when the tunnel is down.

6 Limitations and Future Work

The system has honest limits. Quality degrades on the skip path by design: forcing generation from sparse notes produces valid but thin entries with N/A placeholders, and the paper makes

no claim that skipped entries meet any editorial bar. There is no automated evaluation of entry quality – no rubric scorer, no regression corpus of Raw Notes to gold entries, no measurement of clarification usefulness. The ADMIN_PASSWORD is a single shared secret with no per-user identity, rotation story, or rate limiting beyond the platform’s; the CORS allowlist is two origins wide and edited by redeploy. The PAT expires on a 90-day clock with only a calendar reminder for renewal. Quick tunnels change URLs on every cloudflared restart, so any hardcoded OLLAMA_TUNNEL_URL rots unless moved to a named tunnel. Finally, the paper reports build sizes and latencies from the author’s host and CI history, not from a controlled benchmark.

Natural next steps are: a scored evaluation set for the interview policy (when to ask vs. generate); per-question skip analytics (already partially implemented in the portal) to prune low-value questions; named-tunnel persistence plus health-checked provider failover instead of manual URL pasting; signed provenance (commit SHAs surfaced in the portal) so authors can audit what shipped; and an export path for entries beyond the portfolio grid (RSS, JSON feed, print stylesheet).

7 Conclusion

dkr_Portfolio demonstrates that a personal site can be both statically disciplined and dynamically authored: a Next.js export for speed and free hosting, a Cloudflare Worker for secret-holding orchestration, a self-hosted model for zero-marginal-cost synthesis, a two-phase interview for asking only what is missing, and a strict-in-CI/lenient-at-render contract so automation never breaks the reading experience. The architecture maps cleanly onto established results in Web architecture, edge computing, continuous delivery, interactive learning, and prompting, while remaining small enough for one person to operate. The principal open problem is not plumbing but judgment – measuring and improving what the system chooses to ask – and the harness is structured so that better policies can be introduced without disturbing the commit and deploy interfaces.

References

- Amershi, Saleema, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. 2014. “Power to the People: The Role of Humans in Interactive Machine Learning.” *AI Magazine* 35 (4): 105–20. <https://doi.org/10.1609/aimag.v35i4.2513>.
- Boehm, Barry W. 1988. “A Spiral Model of Software Development and Enhancement.” *Computer* 21 (5): 61–72. <https://doi.org/10.1109/2.59>.
- Fielding, Roy T., and Richard N. Taylor. 2002. “Principled Design of the Modern Web Architecture.” *ACM Transactions on Internet Technology* 2 (2): 115–50. <https://doi.org/10.1145/514183.514185>.
- Fitzgerald, Brian, and Klaas-Jan Stol. 2017. “Continuous Software Engineering: A Roadmap and Agenda.” *Journal of Systems and Software* 123: 176–89. <https://doi.org/10.1016/j.jss.2015.06.063>.
- Kalliamvakou, Eirini, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2014. “The Promises and Perils of Mining GitHub.” *Proceedings of the 11th Working Conference on Mining Software Repositories*, 92–101. <https://doi.org/10.1145/2597073.2597074>.

Knuth, Donald E. 1984. "Literate Programming." *The Computer Journal* 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.

Li, Haoran, Yiran Liu, Xingxing Zhang, Wei Lu, and Furu Wei. 2023. "Tuna: Instruction Tuning Using Feedback from Large Language Models." *Findings of the Association for Computational Linguistics: EMNLP 2023*, 15146–63. <https://doi.org/10.18653/v1/2023.findings-emnlp.1011>.

Madaan, Aman, Shuyan Zhou, Uri Alon, Yiming Yang, and Graham Neubig. 2022. "Language Models of Code Are Few-Shot Commonsense Learners." *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 1384–403. <https://doi.org/10.18653/v1/2022.emnlp-main.90>.

Parnas, David L. 1972. "On the Criteria to Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15 (12): 1053–58. <https://doi.org/10.1145/361598.361623>.

Shi, Weisong, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. "Edge Computing: Vision and Challenges." *IEEE Internet of Things Journal* 3 (5): 637–46. <https://doi.org/10.1109/JIOT.2016.2579198>.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems* 35, 24824–37. <https://doi.org/10.52202/068431-1800>.